

Техническое Задание (PRD): AI-Астростилист

Backend API & Admin Panel

Версия документа: 1.0 (MVP Scope)

Дата создания: 14 марта 2026

Статус: Final for Development

Executive Summary

Бизнес-контекст и ценностное предложение

Современная электронная коммерция сталкивается с критической проблемой снижения эффективности стандартных маркетинговых коммуникаций. Проект **AI-Астростилист** разработан для решения трех взаимосвязанных бизнес-задач:

- 1. Сбор First-Party данных высокой ценности.** Получение дат рождения клиентов и их близких создает основу для персонализированных маркетинговых кампаний с кратно более высокой конверсией [1][2].
- 2. Увеличение эмоциональной вовлеченности.** Астрологический контекст создает уникальный, развлекательный опыт взаимодействия с брендом, выделяющий его среди конкурентов [1][2].
- 3. Рост повторных покупок.** Расширение базы для персонализированных коммуникаций по сценарию "День рождения" значительно увеличивает Customer Lifetime Value [1].

С технической точки зрения, AI-Астростилист представляет собой высоконагруженный Backend-сервис, выступающий связующим звеном (middleware) между пользовательским интерфейсом (Telegram-бот заказчика) и тяжелыми внешними зависимостями (OpenAI API, астрологические провайдеры) [1][2].

Границы ответственности проекта

Входит в scope разработки:

- Backend REST API для интеграции с ботом заказчика
- Асинхронный движок обработки запросов через NATS JetStream
- Система правил маппинга астрологических параметров на каталог товаров (Rule-Engine)
- Web-based Admin Panel для управления каталогом и правилами
- Интеграция с OpenAI API и внешними астрологическими сервисами

НЕ входит в scope разработки (реализуется заказчиком):

- Telegram-бот и весь пользовательский UI
- Интеграция с CRM-системами (retailCRM/amoCRM)
- Платежные интеграции и e-commerce функционал
- Витрина товаров и система каталогизации на стороне магазина

Глоссарий

Термин	Определение
DOB (Date of Birth)	Дата рождения пользователя. Персональные данные, требующие шифрования согласно требованиям GDPR и Ф3-152 [1].
Astro-Trigger	Конкретное астрологическое условие (например, "Венера в Тельце", "Солнце в Овне"), влияющее на стилистические рекомендации.
Rule-Engine	Модуль системы, позволяющий администратору связывать Astro-Triggers с тегами товаров из каталога (например, sport, premium, red) [1].
SuperAdmin	Единственная системная роль в MVP с полным доступом к админ-панели [1].
NATS JetStream	Брокер сообщений с персистентным хранилищем, обеспечивающий гарантированную доставку и retry-логику для асинхронных задач.
Exponential Backoff	Алгоритм прогрессивной задержки при повторных попытках (5s, 30s, 5min, 1h...), предотвращающий перегрузку внешних API.
Dead Letter Queue (DLQ)	Специальный стрим NATS для задач, исчерпавших все попытки retry. Требует ручного вмешательства администратора [3].
Webhook	HTTP-эндпоинт на стороне заказчика, куда Backend отправляет готовые результаты генерации.

Table 1: Основные термины проекта

Архитектура и Технологический стек

Проектирование системы базируется на принципах **низкой связности** (loose coupling), **высокой производительности** и **отказоустойчивости** (fault tolerance).

Backend Core Stack

- **Язык программирования:** Go (Golang) 1.22+
 - Нативная поддержка параллелизма через горутины
 - Оптимален для I/O-bound задач (сетевые запросы, БД)
 - Компиляция в единый статический бинарный файл упрощает деплой
- **HTTP-роутер:** go-chi/chi v5
 - Полная совместимость со стандартной библиотекой net/http
 - Встроенный middleware (Logger, Recoverer, Timeout)
 - Низкие накладные расходы по сравнению с тяжелыми фреймворками
- **База данных:** PostgreSQL 15+
 - Отказ от ORM в пользу сырых SQL-запросов через jackc/pgx/v5/pgxpool
 - Интенсивное использование типа JSONB для гибкого хранения тегов и правил [1]
 - Пул соединений (pgxpool) обеспечивает оптимальную утилизацию ресурсов БД
- **Брокер сообщений:** NATS JetStream
 - Персистентное хранение сообщений на диске
 - Встроенная поддержка Exponential Backoff для retry-логики
 - Высокая пропускная способность (millions msg/sec)
 - Dead Letter Queue для обработки проблемных задач [3]
- **Внешние интеграции:**
 - OpenAI API (ChatGPT) для генерации персонализированных текстов [2]
 - Astro API (провайдер выбирается командой) для расчета натальных карт

Frontend Stack (Admin Panel)

- **Framework:** React 18+ (сборщик Vite)
- **UI-библиотека:** Ant Design или Material-UI
- **State Management:** React Query для управления серверным состоянием
- **HTTP-клиент:** Axios с перехватчиками для автоматической авторизации

- **Языки интерфейса:** Русский (RU) и Английский (EN) [1]

Архитектурные паттерны

Dependency Injection: Все зависимости (pgxpool.Pool, NATS connection, OpenAI client) передаются в HTTP-хэндлеры через структуру AppHandler, что обеспечивает тестируемость и избегание глобальных переменных.

Event-Driven Architecture: Разделение синхронного HTTP-слоя и асинхронной обработки через NATS позволяет API отвечать мгновенно (< 100ms), даже если генерация через OpenAI занимает 10-30 секунд.

Circuit Breaker Pattern: При недоступности внешних API (OpenAI, Astro) система автоматически переключается в режим retry с экспоненциальной задержкой, предотвращая каскадные отказы.

Модель данных (PostgreSQL)

Для минимизации JOIN-операций и обеспечения гибкости используется денормализованная схема с активным применением типа JSONB [1].

Таблица: products (Каталог товаров)

Поле	Тип	Описание
id	UUID	Внутренний первичный ключ
ext_product_id	VARCHAR(255)	ID товара из внешней системы (CRM/магазин)
title	VARCHAR(500)	Название товара
price	NUMERIC(10,2)	Цена в валюте магазина
url	TEXT	Прямая ссылка на страницу товара
images	TEXT[]	Массив URL изображений товара
tags	JSONB	Массив строковых тегов для поиска. Пример: ["red", "sport", "premium"] [1]
created_at	TIMESTAMP	Дата добавления в каталог
updated_at	TIMESTAMP	Дата последнего обновления

Table 2: Структура таблицы products

Индексы:

CREATE INDEX idx_products_tags ON products USING GIN (tags jsonb_path_ops);

GIN-индекс обеспечивает быстрый поиск по тегам через операторы ?, ?|, ?& [4][5].

Таблица: astro_rules (Правила маппинга)

Поле	Тип	Описание
id	UUID	Первичный ключ
name	VARCHAR(255)	Человекочитаемое название правила
astro_condition	JSONB	Астрологическое условие. Пример: {"planet": "Venus", "sign": "Taurus"} [1]
product_tags	JSONB	Массив тегов для поиска товаров. Пример: ["luxury", "comfort", "classic"] [1]
priority	INTEGER	Порядок применения правила (меньше = выше приоритет)
is_active	BOOLEAN	Флаг активности правила
created_at	TIMESTAMP	Дата создания
updated_at	TIMESTAMP	Дата изменения

Table 3: Структура таблицы astro_rules

Таблица: users (Пользователи бота)

Поле	Тип	Описание
id	UUID	Первичный ключ
user_id	VARCHAR(255)	ID пользователя из Telegram/CRM. Уникален.
encrypted_dob	BYTEA	Зашифрованная дата рождения (AES-256-GCM) [1]
consent_given	BOOLEAN	Флаг явного согласия на обработку ПД [1]
created_at	TIMESTAMP	Дата первого взаимодействия
updated_at	TIMESTAMP	Дата последнего обновления

Table 4: Структура таблицы users

Критическое требование безопасности: Поле encrypted_dob НИКОГДА не хранит данные в открытом виде. Шифрование/дешифрование происходит на уровне приложения (Go) с использованием ключа из переменных окружения [1].

Таблица: requests_log (Аудит-лог генераций)

Поле	Тип	Описание
id	UUID	Первичный ключ
request_id	UUID	Публичный идентификатор запроса (возвращается боту)
user_id	VARCHAR(255)	Ссылка на пользователя
scenario	VARCHAR(50)	Тип запроса: "personal_style" или "gift_recommendation" [2]
status	VARCHAR(20)	pending processing retry success failed
attempt_count	INTEGER	Количество попыток обработки
error_reason	TEXT	Текст последней ошибки (для отладки в админке) [1]
result_payload	JSONB	Готовый результат (текст + массив товаров)
created_at	TIMESTAMP	Время создания запроса
updated_at	TIMESTAMP	Время последнего изменения статуса

Table 5: Структура таблицы requests_log

Индексы для производительности:

```
CREATE INDEX idx_requests_status ON requests_log(status, created_at DESC);
```

```
CREATE INDEX idx_requests_user ON requests_log(user_id, created_at DESC);
```

API Contract: Полное описание ЭНДПОИНТОВ

Взаимодействие разделено на две изолированные зоны: **Client API** (для бота заказчика) и **Admin API** (для административной панели) [1].

Client API

Данные эндпоинты должны отвечать с минимальным latency (target: < 100ms P95), делегируя тяжелую работу в фоновые процессы через NATS.

POST /api/v1/astro/profile

Назначение: Получение или расчет базового астрологического профиля по дате рождения [1].

HTTP Request:

POST /api/v1/astro/profile HTTP/1.1
Host: api.astrostylelist.example.com
Content-Type: application/json
Authorization: Bearer {bot_api_key}

Request Body:

```
{  
  "user_id": "telegram_123456789",  
  "date_of_birth": "1990-05-15",  
  "consent_given": true  
}
```

Response (200 OK):

```
{  
  "profile_id": "uuid-v4",  
  "zodiac_sign": "Taurus",  
  "venus_position": "Aries",  
  "mars_position": "Gemini",  
  "dominant_element": "Earth"  
}
```

Response (400 Bad Request):

```
{  
"error": "invalid_date_format",  
"message": "Date of birth must be in ISO 8601 format (YYYY-MM-DD)"  
}
```

Бизнес-логика:

1. Если consent_given = true, дата рождения шифруется и сохраняется в таблице users [1]
2. Делается синхронный запрос к внешнему Astro API для расчета натальной карты
3. Результат кэшируется для последующих запросов (опционально в NATS KV Store)

POST /api/v1/astro/recommend

Назначение: Инициация генерации персональных рекомендаций (образ или подарок) [1][2].

HTTP Request:

POST /api/v1/astro/recommend HTTP/1.1
Host: api.astrostylelist.example.com
Content-Type: application/json
Authorization: Bearer {bot_api_key}

Request Body:

```
{  
"user_id": "telegram_123456789",  
"scenario": "personal_style",  
"gender": "female",  
"preferences": {  
"occasion": "work",  
"budget_range": "medium"  
},  
"webhook_url": "https://bot.customer.com/webhooks/astro-result"  
}
```

Response (202 Accepted):

```
{  
"request_id": "uuid-v4",
```

```
"status": "pending",
"estimated_time_seconds": 30,
"message": "Your recommendation is being generated. Result will be
sent to webhook."
}
```

Бизнес-логика:

1. Создается запись в requests_log со статусом pending
2. Публикуется событие в NATS JetStream стрим astro.events
3. API немедленно возвращает 202 Accepted с request_id
4. Фоновый воркер обрабатывает задачу асинхронно
5. После генерации результат отправляется POST-запросом на webhook_url заказчика

Webhook Payload (отправляется боту после генерации):

```
{
"request_id": "uuid-v4",
"status": "success",
"generated_text": "Венера в Овне наделяет вас смелостью в выборе
стиля...",
"recommended_items": [
{
"product_id": "PROD-123",
"title": "Красная кожаная куртка",
"price": 15990.00,
"url": "https://shop.example.com/products/123",
"images": ["https://cdn.example.com/img1.jpg"],
"reason": "Яркий красный цвет соответствует энергии Овна"
},
{
"product_id": "PROD-456",
"title": "Спортивные кроссовки",
"price": 8990.00,
"url": "https://shop.example.com/products/456",
"images": ["https://cdn.example.com/img2.jpg"],
"reason": "Активный стиль для вашей динамичной натуры"
}
]
}
```

POST /api/v1/feedback

Назначение: Сохранение обратной связи от пользователя для улучшения промптов [2].

HTTP Request:

POST /api/v1/feedback HTTP/1.1

Host: api.astrostylelist.example.com

Content-Type: application/json

Authorization: Bearer {bot_api_key}

Request Body:

```
{
  "request_id": "uuid-v4",
  "rating": "positive",
  "comment": "Отличная подборка, все понравилось!"
}
```

Response (200 OK):

```
{
  "message": "Feedback saved successfully"
}
```

Admin API

Все Admin-эндпоинты защищены авторизацией (JWT Bearer Token или Basic Auth). Доступны только роли SuperAdmin [1].

POST /api/v1/admin/catalog/import

Назначение: Загрузка и обновление каталога товаров через CSV-файл [1].

HTTP Request:

POST /api/v1/admin/catalog/import HTTP/1.1

Host: api.astrostylelist.example.com

Content-Type: multipart/form-data

Authorization: Bearer {admin_jwt_token}

Form Data:

- file: CSV-файл с товарами

- `update_strategy`: "replace" | "merge" (перезапись или дополнение)

Ожидаемая структура CSV:

product_id,title,price,url,images,tags

PROD-123,"Красная куртка",15990.00,"https://...",<https://cdn.../img1.jpg>,"red,leather,sport"

PROD-456,"Джинсы",5990.00,"https://...",<https://cdn.../img2.jpg>,"blue,casual,denim"

Response (200 OK):

```
{
  "imported": 156,
  "updated": 23,
  "errors": 2,
  "error_details": [
    {"row": 45, "reason": "Invalid price format"},
    {"row": 78, "reason": "Missing required field: url"}
  ]
}
```

Бизнес-логика:

1. Парсинг CSV с валидацией обязательных полей
2. Нормализация данных (split тегов по запятым, валидация URL)
3. Batch Insert/Update в таблицу products через pgxpool
4. Транзакционная запись: либо весь батч, либо откат при критических ошибках

GET /api/v1/admin/products

Назначение: Получение списка товаров для просмотра в админке.

HTTP Request:

GET /api/v1/admin/products?page=1&limit=50&tag=red HTTP/1.1

Host: api.astrostylelist.example.com

Authorization: Bearer {admin_jwt_token}

Query Parameters:

- page: номер страницы (default: 1)
- limit: количество на странице (default: 50, max: 200)
- tag: фильтр по тегу (опционально)
- search: поиск по названию (опционально)

Response (200 OK):

```
{
  "total": 1523,
  "page": 1,
  "limit": 50,
  "items": [
    {
      "id": "uuid",
      "ext_product_id": "PROD-123",
      "title": "Красная кожаная куртка",
      "price": 15990.00,
      "tags": ["red", "leather", "sport"],
      "images": ["https://cdn.example.com/img1.jpg"]
    }
  ]
}
```

POST /api/v1/admin/rules

Назначение: Создание нового правила маппинга [1].

HTTP Request:

POST /api/v1/admin/rules HTTP/1.1
 Host: api.astrostylelist.example.com
 Content-Type: application/json
 Authorization: Bearer {admin_jwt_token}

Request Body:

```
{
  "name": "Венера в Тельце → Роскошь и комфорт",
  "astro_condition": {
    "planet": "Venus",
    "sign": "Taurus"
  },
  "product_tags": ["luxury", "comfort", "classic", "high-quality"],
  "priority": 10,
}
```

```
"is_active": true
}
```

Response (201 Created):

```
{
  "id": "uuid-v4",
  "message": "Rule created successfully"
}
```

GET /api/v1/admin/rules

Назначение: Получение списка всех правил.

HTTP Request:

GET /api/v1/admin/rules HTTP/1.1
Host: api.astrostylist.example.com
Authorization: Bearer {admin_jwt_token}

Response (200 OK):

```
{
  "rules": [
    {
      "id": "uuid-1",
      "name": "Венера в Тельце → Роскошь",
      "astro_condition": {"planet": "Venus", "sign": "Taurus"},
      "product_tags": ["luxury", "comfort"],
      "priority": 10,
      "is_active": true
    }
  ]
}
```

PUT /api/v1/admin/rules/{id}

Назначение: Редактирование существующего правила.

HTTP Request:

PUT /api/v1/admin/rules/{uuid} HTTP/1.1
Host: api.astrostylist.example.com
Content-Type: application/json
Authorization: Bearer {admin_jwt_token}

Request Body: (аналогичен POST, но с обновляемыми полями)

DELETE /api/v1/admin/rules/{id}

Назначение: Удаление правила.

HTTP Request:

DELETE /api/v1/admin/rules/{uuid} HTTP/1.1

Host: api.astrostylelist.example.com

Authorization: Bearer {admin_jwt_token}

Response (204 No Content)

GET /api/v1/admin/logs

Назначение: Получение логов генераций для дашборда мониторинга [1].

HTTP Request:

GET /api/v1/admin/logs?status=failed&page=1&limit=100 HTTP/1.1

Host: api.astrostylelist.example.com

Authorization: Bearer {admin_jwt_token}

Query Parameters:

- status: фильтр по статусу (pending, retry, failed, success)
- date_from, date_to: временной диапазон
- page, limit: пагинация

Response (200 OK):

```
{
  "total": 342,
  "page": 1,
  "items": [
    {
      "request_id": "uuid",
      "user_id": "telegram_123",
      "status": "failed",
      "error_reason": "OpenAI API: 429 Rate Limit Exceeded",
      "attempt_count": 5,
      "created_at": "2026-03-14T10:30:00Z",
      "updated_at": "2026-03-14T10:45:00Z"
    }
  ]
}
```

```
}  
]  
}
```

Асинхронный процесс обработки (NATS JetStream Workflow)

Ключевая особенность архитектуры — разделение синхронного HTTP-слоя и тяжелой обработки. Это обеспечивает низкий latency API и высокую отказоустойчивость при проблемах с внешними сервисами.

Пошаговый workflow генерации рекомендации

1. **HTTP Request:** Бот вызывает POST /api/v1/astro/recommend
2. **Publish to NATS:** Хэндлер создает запись в requests_log (status = pending), публикует событие в JetStream стрим astro.events и возвращает 202 Accepted
3. **Consumer Picks Up:** Фоновая горутина (NATS Consumer) забирает сообщение, обновляет статус на processing
4. **External API Calls:**
 - Запрос к Astro API для получения натальной карты (если не кэширована)
 - SQL-запрос к PostgreSQL для поиска товаров через JSONB-операторы [1][4]
 - Формирование промпта и запрос к OpenAI API [2]
5. **Error Handling:** Если любой из внешних сервисов возвращает ошибку:
 - Воркер пишет error_reason в requests_log
 - Обновляет статус на retry
 - Отправляет NAK в NATS (без параметров)
 - NATS применяет Exponential Backoff и вернет задачу позже [3]
6. **Success Path:** При успешном ответе от OpenAI:
 - Парсинг JSON-ответа
 - Сохранение result_payload в requests_log (status = success)
 - Отправка POST-запроса на webhook_url бота с готовым результатом

- Отправка ACK в NATS (задача завершена)
7. **Dead Letter Queue:** Если задача исчерпала все попытки retry (например, 5 попыток), NATS автоматически перемещает ее в специальный DLQ-стрим `astro.dlq` [3]
- В `requests_log` статус меняется на `failed`
 - Администратор видит эти задачи в разделе "Логи" админки [1]
 - Требуется ручное расследование причины (например, истек API-ключ OpenAI)

Конфигурация NATS JetStream Consumer

```
consumerConfig := jetstream.ConsumerConfig{
    Durable: "astro-recommendations-worker",
    FilterSubject: "astro.events.recommend",
    AckPolicy: jetstream.AckExplicitPolicy,
    MaxDeliver: 5, // Максимум 5 попыток
    BackOff: []time.Duration{
        5 * time.Second, // 1-я повторная попытка через 5 сек
        30 * time.Second, // 2-я через 30 сек
        5 * time.Minute, // 3-я через 5 мин
        1 * time.Hour, // 4-я через 1 час
    },
}
```

SQL-запрос поиска товаров по тегам

После получения списка тегов из Rule-Engine (например, `["luxury", "comfort"]`), воркер выполняет следующий запрос:

```
SELECT id, ext_product_id, title, price, url, images, tags
FROM products
WHERE tags ?| ARRAY['luxury', 'comfort'] -- JSONB operator: any of
tags match
AND price BETWEEN $1 AND $2 -- Budget filtering
ORDER BY RANDOM() -- Добавляем разнообразие
LIMIT 10;
```

Оператор `?|` проверяет пересечение JSONB-массива с массивом строк [4][5]. GIN-индекс делает этот запрос очень быстрым даже на миллионах товаров.

Требования к UI Админ-панели

Интерфейс строится по принципу **функциональности и минимализма** [1]. Дизайн нейтральный, не перегруженный, с фокусом на рабочих задачах администратора.

Экран 1: Dashboard (Главная / Мониторинг)

Компоненты интерфейса:

- **Карточки метрик (вверху):**
 - Всего запросов за последние 24 часа
 - Процент успешных генераций (Success Rate)
 - Количество задач в статусе retry
 - Количество задач в DLQ (требуют внимания)
- **Таблица логов (requests_log):**
 - Колонки: Request ID, User ID, Scenario, Status, Error Reason, Created At
 - Цветовая индикация статусов: pending (синий), processing (желтый), success (зеленый), failed (красный)
 - Фильтры: по статусу, временному диапазону
 - Кнопка "Export to CSV" для выгрузки логов
- **Детальный просмотр:** Клик по строке открывает модальное окно с полной информацией о запросе, включая сгенерированный текст и список товаров (если status = success)

Экран 2: Catalog Management (Управление каталогом)

Компоненты интерфейса:

- **Зона импорта (вверху):**
 - Drag-and-drop область для загрузки CSV-файла [1]
 - Выбор стратегии: "Replace all" (полная перезапись) или "Merge" (обновление существующих)
 - Кнопка "Upload" с индикатором прогресса
 - После загрузки: сводка "Imported: 156, Updated: 23, Errors: 2" с возможностью скачать отчет об ошибках
- **Таблица товаров (ниже):**

- Колонки: Image (миниатюра), Title, Price, Tags, Actions
- Поиск по названию (полнотекстовый)
- Фильтр по тегам (multiselect dropdown)
- Пагинация (50-100-200 элементов на странице)
- **Просмотр товара:** Клик на строку открывает карточку товара с полными данными и возможностью перейти по URL в магазин

Экран 3: Rule-Engine Editor (Редактор правил маппинга)

Компоненты интерфейса:

- **Список правил (левая панель):**
 - Карточки существующих правил с названием, условием и тегами
 - Индикатор активности (зеленый = active, серый = inactive)
 - Drag-and-drop для изменения приоритета применения правил [1]
 - Кнопки: Edit, Deactivate, Delete
- **Форма создания/редактирования (правая панель):**
 - Поле "Rule Name" (человекочитаемое название)
 - Секция "Astro Condition" (условие):
 - * Dropdown "Planet" (Sun, Moon, Venus, Mars...)
 - * Dropdown "Sign" (Aries, Taurus, Gemini...)
 - * Возможность добавить несколько условий (OR-логика)
 - Секция "Product Tags" (теги товаров):
 - * Multiselect с автодополнением из существующих тегов в БД
 - * Возможность ввести кастомные теги вручную
 - Slider "Priority" (1-100, где 1 = highest priority)
 - Checkbox "Active" (включить/выключить правило)
 - Кнопка "Save Rule" или "Update Rule"

Пример UX-сценария:

Администратор видит, что по правилу "Венера в Овне" генерируются слишком агрессивные рекомендации. Он открывает редактор, находит это правило, убирает тег

"aggressive" из списка Product Tags, добавляет "bold" и "confident", сохраняет изменения. С этого момента все новые генерации будут использовать обновленное правило.

Безопасность и Compliance

Шифрование персональных данных (DOB)

Алгоритм: AES-256-GCM (Galois/Counter Mode) с аутентификацией [1].

Ключ шифрования: Передается через переменную окружения ENCRYPTION_KEY (длина 32 байта, base64-encoded). НИКОГДА не хранится в коде или конфигурационных файлах в репозитории.

Процесс шифрования (в Go):

```
func EncryptDOB(plaintext string, key []byte) ([]byte, error) {  
    block, err := aes.NewCipher(key)  
    if err != nil {  
        return nil, err  
    }
```

```
    gcm, err := cipher.NewGCM(block)  
    if err != nil {  
        return nil, err  
    }  
  
    nonce := make([]byte, gcm.NonceSize())  
    if _, err := io.ReadFull(rand.Reader, nonce); err != nil {  
        return nil, err  
    }  
  
    ciphertext := gcm.Seal(nonce, nonce, []byte(plaintext), nil)  
    return ciphertext, nil
```

```
}
```

Процесс дешифрования: Происходит только при явном запросе на удаление данных или для отправки в аналитику (с согласия пользователя).

Управление согласием (Consent Management)

Принцип: Opt-in по умолчанию [1].

- Если бот передает `consent_given = false`, дата рождения НЕ сохраняется в БД
- Астрологический профиль рассчитывается, но хранится только в оперативной памяти (in-memory cache) на время сессии
- После генерации рекомендации все данные уничтожаются
- Если позже пользователь дает согласие, создается запись в таблице `users` с зашифрованным DOB

Право на забвение (Right to be Forgotten)

Реализация:

```
-- При получении запроса на удаление данных пользователя:  
BEGIN;  
DELETE FROM users WHERE user_id = $1;  
DELETE FROM requests_log WHERE user_id = $1;  
COMMIT;
```

Физическое удаление без возможности восстановления.

Транзакция гарантирует атомарность операции.

Идемпотентность операций

Все операции через NATS должны быть идемпотентными. Если воркер обработал задачу, но не успел отправить ACK до сбоя сети, NATS может переотправить сообщение. Воркер проверяет наличие готового `result_payload` в `requests_log` перед повторной обработкой:

```
// Проверка идемпотентности  
var existingResult string  
err := db.QueryRow(ctx,  
"SELECT result_payload FROM requests_log WHERE request_id = $1
```

```
AND status = 'success',
requestID).Scan(&existingResult)

if err == nil {
// Результат уже есть, отправляем ACK без повторной обработки
msg.Ack()
return
}
```

Производительность и масштабирование

Target Performance SLAs

Метрика	Целевое значение
Latency HTTP API (P95)	< 100ms
Latency HTTP API (P99)	< 200ms
Throughput API	1000 req/s на 1 инстанс
Время генерации рекомендации (end-to-end)	15-45 секунд (зависит от OpenAI)
Доступность (Availability)	99.9% (допустимо ~43 мин downtime/месяц)

Table 6: Service Level Agreements

Горизонтальное масштабирование

Backend API: Stateless-сервисы. Можно запускать несколько инстансов за балансировщиком нагрузки (Nginx, AWS ALB). Все состояние хранится в PostgreSQL и NATS.

NATS Workers: Можно запускать N воркеров параллельно. NATS JetStream автоматически распределяет задачи между Consumer-группами.

PostgreSQL: Вертикальное масштабирование на начальном этапе. При росте нагрузки > 10K RPS — внедрение Read Replicas для аналитических запросов админки.

Оптимизация базы данных

- **Connection Pooling:** pgxpool с настройкой MaxConns = 20-50 (зависит от ресурсов сервера)
 - **Prepared Statements:** Переиспользование подготовленных запросов для уменьшения парсинга SQL
 - **GIN-индексы для JSONB:** Критически важны для быстрого поиска по тегам [4][5]
 - **VACUUM:** Регулярная очистка устаревших версий строк (автоматически через autovacuum)
-

Тестирование

Unit Tests (Golang)

- Покрытие > 80% для критических модулей (шифрование, Rule-Engine, JSONB-запросы)
- Использование mock-интерфейсов для pgxpool и NATS (библиотека gomock или testify)
- Изоляция внешних зависимостей (OpenAI, Astro API) через HTTP-моки (httpmock)

Integration Tests

- Тестирование полного workflow: HTTP → NATS → Worker → Webhook
- Использование testcontainers для запуска PostgreSQL и NATS в Docker
- Проверка корректности retry-логики при имитации сбоев OpenAI (429, 500)

Load Testing

- Инструменты: k6.io или Apache JMeter
 - Сценарии: 1000 одновременных POST /recommend запросов
 - Метрики: latency (P50, P95, P99), throughput, error rate
-

Развертывание и DevOps

Containerization

Dockerfile для Backend:

```
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o /astro-backend
./cmd/api
```

```
FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=builder /astro-backend .
EXPOSE 8080
CMD ["/astro-backend"]
```

Docker Compose (для локальной разработки):

```
version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: astrostylist
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    ports:
      - "5432:5432"

  nats:
    image: nats:latest
    command: ["-js", "-sd", "/data"]
    ports:
      - "4222:4222"

  backend:
    build: .
```

environment:

DATABASE_URL: postgres://admin:secret@postgres:5432/astrostylist

NATS_URL: nats://nats:4222

ENCRYPTION_KEY: your-32-byte-base64-key

depends_on:

- postgres

- nats

ports:

- "8080:8080"

Миграции базы данных

Использование инструмента **golang-migrate**:

```
migrate create -ext sql -dir migrations -seq create_products_table
```

```
migrate -database "postgres://..." -path ./migrations up
```

CI/CD Pipeline

1. GitHub Actions / GitLab CI:

- Запуск unit-тестов на каждый commit
- Сборка Docker-образа
- Push образа в Container Registry (Docker Hub, AWS ECR)

2. Deployment:

- Production: Kubernetes (Helm charts) или AWS ECS
- Staging: Автоматический деплой на каждый merge в main branch

Мониторинг и Observability

Логирование

- **Структурированные логи:** JSON-формат (библиотека zerolog или zap)
- **Уровни логов:** DEBUG (только dev), INFO (production), ERROR (критические ошибки)
- **Централизованное хранение:** ELK Stack (Elasticsearch + Logstash + Kibana) или AWS CloudWatch

Метрики (Prometheus)

- HTTP-метрики: request_count, request_duration, error_rate (по эндпоинтам)
- NATS-метрики: messages_processed, messages_failed, backoff_triggered
- БД-метрики: connection_pool_size, query_duration

Алерты

- Error Rate > 5% за последние 5 минут → уведомление в Slack/PagerDuty
- Задач в DLQ > 10 → уведомление администратору
- Latency P95 > 500ms → предупреждение о деградации производительности

Приложение: Примеры промптов для OpenAI

Промпт для сценария "Личный стиль"

Ты — эксперт-стилист с глубокими знаниями астрологии.

Пользователь:

- Дата рождения: {dob}
- Пол: {gender}
- Астропрофиль: Солнце в {sun_sign}, Венера в {venus_sign}

Задача: На основе астрологического профиля создай вдохновляющую рекомендацию стиля (150-200 слов). Упомяни, как положение Венеры влияет на эстетические предпочтения.

Ниже список подходящих товаров. Выбери 2-4 наиболее соответствующих и для каждого напиши краткое обоснование (1 предложение).

Товары:

{products_json}

Верни ответ в формате JSON:

```
{
  "text": "...",
  "items": [
    {"product_id": "...", "reason": "..."},
    ...
  ]
}
```

Промпт для сценария "Идеальный подарок"

Пользователь ищет подарок для близкого человека:

- Получатель: {gender}, дата рождения {dob}
- Астропрофиль получателя: {zodiac_sign}, Венера в {venus_sign}
- Повод: {occasion} (День рождения / Без повода)
- Бюджет: {budget_range}

Создай персонализированную рекомендацию подарка (120-150 слов), объясняя связь между астрологией получателя и предложенными товарами.

Товары: {products_json}

Верни JSON-ответ аналогично предыдущему сценарию.

Заключение и следующие шаги

Данное техническое задание охватывает все ключевые аспекты реализации MVP-версии сервиса AI-Астростилист.

Рекомендуемые этапы разработки:

1. **Phase 1 (Weeks 1-3):** Настройка инфраструктуры (PostgreSQL, NATS), создание базовой структуры Backend на Go, реализация эндпоинтов Client API
2. **Phase 2 (Weeks 4-6):** Разработка Rule-Engine, интеграция с OpenAI и Astro API, реализация асинхронного workflow через NATS

3. **Phase 3 (Weeks 7-9):** Разработка Admin Panel (React), реализация экранов Catalog Management и Rule-Engine Editor
4. **Phase 4 (Weeks 10-12):** Интеграционное тестирование, load testing, подготовка к production deployment

После MVP (Post-Launch):

- Сбор метрик конверсии и А/В-тестирование различных промптов для OpenAI
- Внедрение машинного обучения для автоматической оптимизации Rule-Engine на основе feedback пользователей
- Расширение на другие каналы коммуникации (WhatsApp, Viber, веб-виджет)

References

- [1] Brif-dizain-razrabotka-rasshirennyi_ILINE_GROUP.docx (Исходный бриф заказчика)
- [2] AI-astrotilist-2.jpg (Блок-схема бизнес-процесса)
- [3] NATS JetStream Dead Letter Queue: Complete Implementation Guide. StreamTrace. <https://streamtrace.io/articles/nats-jetstream-dead-letter-queue-implementation/>
- [4] How to Query and Index JSONB Efficiently in PostgreSQL. OneUptime Blog. <https://oneuptime.com/blog/post/2026-01-26-jsonb-querying-indexing-postgresql/>
- [5] Using JSONB in PostgreSQL: How To Store & Index JSON Data. ScaleGrid Blog. <https://scalegrid.io/blog/using-jsonb-in-postgresql-how-to-effectively-store-index-json-data-in-postgresql/>